

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Towards AI · Following

How to Fine-Tune an LLM: SFT, LoRA, QLoRA and DPO Explained

This blog post discusses the details of what finetuning is, why it's needed, and how we can finetune an LLM model with practical examples.

20 min read · 1 day ago



Anubhav Mandarwal

Follow



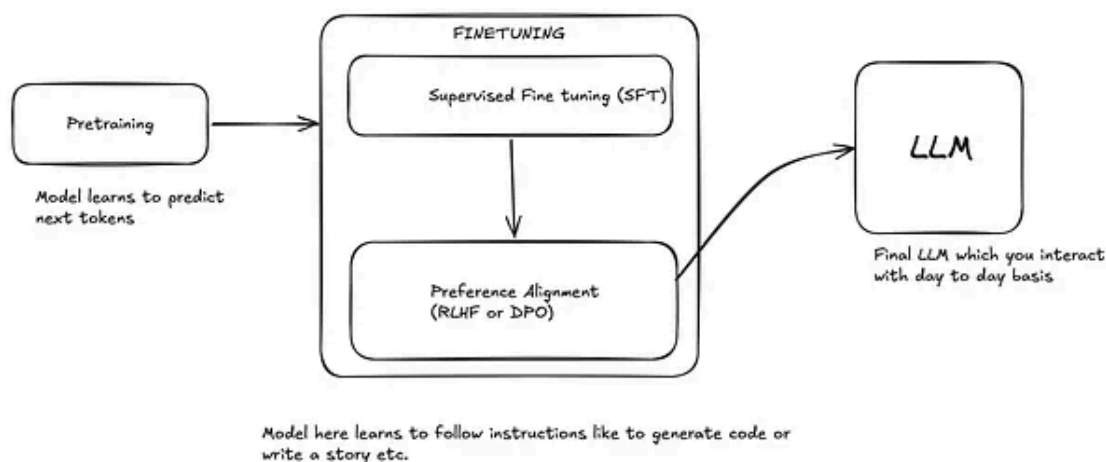
Listen



Share

... More

The fine-tuning is what brings life to the LLM model. It's a technique to make models adapt to a specific task, such as coding, writing poems or songs, classifying objects in an image, etc. A typical lifecycle of LLM training is depicted below.



In the pretraining model, it only learns to predict the next token, but to follow instructions and be able to interact with us like a chatbot, they are finetuned specifically. This helps to increase model domain-specific knowledge like Math, Coding or Image generation, etc.

Finetuning VS RAG

A common area of confusion is when to finetune a model and when to use RAG for the specific task. Say you want the model to answer specific custom questions on which the model is not trained, so should we use finetuning or using RAG be sufficient?

Best advice I've received after reading from various sources and experimenting myself with AI Engineering is that *Fine-tuning is useful when you want to change how the model behaves, reasons, formats responses, or follows a domain-specific pattern. RAG is usually better when you want the model to answer using external or frequently changing knowledge.*

Supervised Fine-Tuning (SFT)

SFT refines the model capability by using pairs of instructions and corresponding answers. We expose the model to these desired input-output patterns and shape the LLM behaviour to align with specific domain tasks like coding, Math, chemistry, etc.

Data preparation

For SFT fine-tuning, your dataset usually becomes many input -> output examples, where:

- Input: the instruction, question, prompt, or context that the model will see
- Output: the ideal answer you want the model to learn to produce

```
{  
  "input": "Explain supervised fine-tuning in simple terms.",  
  "output": "Supervised fine-tuning is the process of training a pretrained lan  
}
```

For chat-based SFT, the same idea is often represented as messages:

```
{  
  "messages": [  
    {  
      "role": "user",  
      "content": "Explain supervised fine-tuning in one paragraph."  
    },  
    {  
      "role": "assistant",  
      "content": "Supervised fine-tuning is a training step where a pretrained  
    }  
  ]  
}
```

We generate many such pairs and then train the pretrained model(base model) on these pairs in Finetuning.

HuggingFace contains a bunch of such datasets specifically for finetuning the base model. In one of my recent projects, where I created my own LLM on the GPT-2 architecture, I used `TinyStoriesInstruct` a dataset for instruction finetuning. If you are interested in that, check out my repository.

<https://github.com/VrityaCodeRishi/Vritya-Tiny-163M-1>

Mostly, these data can also be generated with the use of the LLM itself; we can generate a large synthetic dataset with strong models like opus-4.7 as of writing and then use that dataset to finetune the model. The quality of the data determines how good the model will be finetuned for our specific needs.

I generated 1000 pairs of SFT finetuning data so my model can behave like Ayanokoji Kiyotaka from Classroom of the Elite. This is another fun project I did with Finetuning, which we will be understanding in this blog.

We will be understanding this project in our SFT and DPO topics part of the blog later.

curious-techie/PersonaLM-Ayanokoji-8B · Hugging Face

We're on a journey to advance and democratise artificial intelligence through open source and open science.

huggingface.co

The prompt I used to generate the synthetic dataset from Claude

Prompt to use for generating data:

"You are Kiyotaka Ayanokoji from Classroom of the Elite.

Respond to the following question in his exact style –

cold, analytical, philosophical, minimal emotion,

treats everything as a logical observation.

Question: [question here]"

Generate 1000 examples this way covering:

- Life advice questions
- Philosophy questions
- Human nature questions
- Strategy and competition
- Emotions and relationships
- Success and failure

I need this data for finetuning using LORA so keep the format in jsonl I will be using
MODEL_NAME = "unsloth/Llama-3.1-8B-Instruct"

Instruction dataset formats

They are stored in particular format to organise instructions and answers so they can be represented as dictionaries.

Here are a few common **instruction dataset formats** used for SFT fine-tuning.

1. Alpaca Format

One of the most common formats. Each example is a dictionary with `instruction`, optional `input`, and `output`.

```
{  
  "instruction": "Explain supervised fine-tuning in simple terms.",  
  "input": "",  
  "output": "Supervised fine-tuning is the process of training a pretrained model on a specific task using a dataset of input-output pairs."  
}
```

With extra input/context:

```
{
  "instruction": "Summarize the following paragraph.",
  "input": "Supervised fine-tuning is commonly used after pretraining to adapt a",
  "output": "Supervised fine-tuning adapts a pretrained model to follow instruct
}
```

2. ShareGPT / Chat Format

This format stores conversations as a list of turns.

```
{
  "conversations": [
    {
      "from": "human",
      "value": "What is supervised fine-tuning?"
    },
    {
      "from": "gpt",
      "value": "Supervised fine-tuning is a training process where a pretrained mode
    }
  ]
}
```

Multi-turn example:

```
{
  "conversations": [
    {
      "from": "human",
      "value": "What is SFT?"
    },
    {
      "from": "gpt",
      "value": "SFT stands for supervised fine-tuning."
    },
    {
      "from": "human",
```

```
  "value": "Why is it used?"
},
{
  "from": "gpt",
  "value": "It is used to teach a pretrained model how to follow instructions and"
}
]
```

3. OpenAI Chat Messages Format

Commonly used for chat model fine-tuning. Each training example has a `messages` list.

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful AI assistant."
    },
    {
      "role": "user",
      "content": "Explain SFT in one sentence."
    },
    {
      "role": "assistant",
      "content": "Supervised fine-tuning teaches a pretrained model to respond correctly"
    }
  ]
}
```

4. Dolly Format

Used by Databricks Dolly-style instruction datasets.

```
{
  "instruction": "Classify the sentiment of the sentence.",
  "context": "The product quality is excellent and delivery was fast.",
}
```

```
"response": "Positive"
}
```

Another example:

```
{
  "instruction": "Write a short email requesting a meeting.",
  "context": "",
  "response": "Hi, I would like to schedule a meeting to discuss the project upc
}
```

5. FLAN / Prompt-Completion Format

A simpler format where the prompt contains the full task and the completion contains the answer.

```
{
  "prompt": "Translate the following sentence to French: I am learning machine l",
  "completion": "J'apprends l'apprentissage automatique."
}
```

Another example:

```
{
  "prompt": "Question: What is overfitting in machine learning?\nAnswer:",
  "completion": "Overfitting happens when a model memorizes training data too cl
}
```

6. Instruction-Input-Response Format

This is similar to Alpaca, but some datasets use `response` instead of `output`.

```
{  
  "instruction": "Generate a title for the blog post.",  
  "input": "A beginner-friendly guide to supervised fine-tuning and instruction  
  "response": "A Beginner's Guide to Supervised Fine-Tuning Datasets"  
}
```

Most instruction datasets store each training example as a dictionary.

The keys may differ across formats, but the goal is the same: represent the user's task and the ideal model response.

Common key mappings:

```
Alpaca: instruction + input -> output  
Dolly: instruction + context -> response  
OpenAI chat: messages[user] -> messages[assistant]  
ShareGPT: human turns -> gpt turns  
Prompt-completion: prompt -> completion
```

Chat templates

Chat templates define how a conversation is converted into the exact text format the model sees during training or inference.

In chat fine-tuning, your dataset may be stored as structured messages like this:

```
{  
  "messages": [  
    {  
      "role": "system",  
      "content": "You are a helpful assistant."  
    },  
    {  
      "role": "user",  
      "content": "Explain supervised fine-tuning in simple terms."  
    },  
    {  
      "role": "assistant",  
      "content": "Supervised fine-tuning teaches a pretrained model using examples."  
    }  
  ]  
}
```



```
]
}
```

But the model does not directly read dictionaries. Before training, the tokenizer applies a chat template that converts these messages into a single formatted string.

Example rendered chat template:

```
<|system|>
You are a helpful assistant.
<|user|>
Explain supervised fine-tuning in simple terms.
<|assistant|>
Supervised fine-tuning teaches a pretrained model using examples of user inputs
```

Different models use different chat templates. This matters because the model was trained to understand a specific structure.

Example 1: Simple Role-Based Template

```
System: You are a helpful assistant.
User: Explain SFT in one sentence.
Assistant: SFT trains a pretrained model on examples of prompts and ideal responses
```

This is easy to understand, but real models often use special tokens instead of plain labels.

Example 2: ChatML-Style Template

```
<|im_start|>system
You are a helpful assistant.
<|im_end|>
<|im_start|>user
What is supervised fine-tuning?
```

```
<|im_end|>
<|im_start|>assistant
Supervised fine-tuning is the process of training a pretrained model on instruc
<|im_end|>
```

Here, <|im_start|> and <|im_end|> mark message boundaries.

Example 3: Llama-Style Template

```
<s>[INST] Explain supervised fine-tuning in simple terms. [/INST]
Supervised fine-tuning teaches a pretrained model to follow instructions by tra
```

Example 4: Multi-Turn Chat Template

Structured dataset:

```
{
  "messages": [
    {
      "role": "user",
      "content": "What is SFT?"
    },
    {
      "role": "assistant",
      "content": "SFT stands for supervised fine-tuning."
    },
    {
      "role": "user",
      "content": "Why is it useful?"
    },
    {
      "role": "assistant",
      "content": "It helps adapt a pretrained model to follow instructions and"
    }
  ]
}
```

Rendered as text:

```
<|user|>
What is SFT?
<|assistant|>
SFT stands for supervised fine-tuning.
<|user|>
Why is it useful?
<|assistant|>
It helps adapt a pretrained model to follow instructions and produce responses
```

A chat template is the bridge between a structured conversation dataset and the plain text sequence used to train a language model. It tells the tokenizer how to place system, user, and assistant messages using the special tokens expected by that model.

Parameter Efficient Fine-Tuning Techniques(PEFT)

Now let's look at how models are finetuned in practice.

There are broadly three most used right now as of writing for SFT.

1. Fine tuning the layers of model directly
2. LORA (Freezing model weights and training additional weights then combine those on top of the model weights).
3. QLORA

We will discuss each one by one

1. Fine tuning the layers of model directly

This approach updates the actual parameters, or weights, of the pretrained model. During training, the model sees the new supervised dataset, calculates the error between its predicted output and the expected output, and then uses backpropagation to adjust its internal weights.

In simple terms, we are directly modifying the model's memory. The model is no longer exactly the same as the original pretrained model after this process. It has been reshaped to perform better on the new task or domain.

For example, if we finetune a general language model on medical question-answering data, the model may become better at answering medical questions.

Similarly, if we finetune it on customer support conversations, it may become better at responding like a support assistant.

However, this approach can be destructive if not done carefully. Since the original weights are being changed, the model may lose some of the general knowledge or behavior it learned during pretraining. This problem is often called catastrophic forgetting.

Catastrophic forgetting happens when the model becomes too specialized on the new training data and forgets how to handle broader tasks. For example, a model finetuned heavily on legal contracts may become very good at legal text but worse at casual conversation, reasoning, or general writing.

This is why direct full fine-tuning usually requires careful choices:

- a high-quality dataset
- a suitable learning rate
- enough training examples
- validation checks
- avoiding too many training epochs
- monitoring whether the model is losing general ability

Direct fine-tuning can be powerful, but it is also expensive. Large models contain billions of parameters, so updating all layers requires significant GPU memory, compute, and storage. The final finetuned model is also a full copy of the original model with modified weights.

Full fine-tuning changes the original model weights directly. This can make the model better at a specific task, but it can also overwrite some of the general knowledge learned during pretraining.

Before fine-tuning:

The model can answer many general questions reasonably well.

After full fine-tuning on company support tickets:

The model becomes better at answering company-specific support queries.

Possible downside:

If trained too aggressively, the model may become worse at general tasks because

I used this approach when building my own 163M parameter model pretrained on TinyStories dataset and then finetuned on TinyStories instruct dataset. Given a scenario or object it is able to generate a short story.

Common advice when we are doing such kind of partial finetuning by directly changing model

Partial fine-tuning:

Freeze most layers.

Train the last 1-4 transformer blocks + final norm + task head.

In that we did a *partial finetuning* where we finetuned only these layers of gpt-2

- last 2 transformer blocks
- final normalization layer
- language modeling head

Rest of the layers we froze so they did not get updated during finetuning training.

```
for param in model.parameters(): ## Freezing all the layers first
    param.requires_grad = False

## Now selectively making few layers trainable again (unfreezing).

for block in model.transformer_blocks[-2:]:
    for param in block.parameters():
        param.requires_grad = True

for param in model.final_norm.parameters():
    param.requires_grad = True
```

```
for param in model.linear_head.parameters():  
    param.requires_grad = True
```

Pretrain GPT model on TinyStories
↓
Load the model
↓
Freeze most pretrained layers
↓
Unfreeze final layers / task head
↓
Train on task-specific supervised data
↓
Save task-specific finetuned weights

The reasoning for specifically enabling the last transformer layers and output and final norm layer is as

Early layers:
learn basic token patterns, syntax, word relationships

Middle layers:
learn broader language structure and reusable features

Later layers:
shape the representation for the final task/output

We freeze most of the model to preserve the general knowledge learned during pretraining, and only update the last few layers so the model can adapt to the new task.

Here is the full finetuning script if you are interested

Vritya-Tiny-163M-1/instruction-finetuning.py at main · VrityaCodeRishi/Vritya-Tiny-163M-1

The LLM model with gpt-2 as base model architecture from ground up. Finetuned for spam classification and producing...

github.com

Now let's come to the industry practice for finetuning as of writing which is to freeze the entire model weights and then train additional parameters in finetuning and merge them into the model.

2. Adapter Based Techniques

These are called adapter based techniques cause they allow adding few extra trainable parameters to the model. Here original model weights are untouched

Original model weight: **frozen**
adapter weights: **trainable**

Low-Rank Adaptation in AI (LORA)

It's a parameter efficient fine-tuning technique created to solve computational challenges. It allows finetuning LLM model with significantly reduced computational resources.

We will be understanding our Ayanokoji Kiyotaka project whose link I shared in starting of the blog.

At it's core LORA introduces two additional weights matrices let's call them A and B.

Consider original model weights as W (After pretraining).

These matrices are chosen in such a way when we do product of them we get the same shape as original model weights W.

Now in finetuning we train these two matrices A , B only and keep the original model weights W frozen.

The new weights of the model become W'

$$W' = W + BA$$

LORA has two main hyperparameters

- **Rank (r)** : Size of the LORA matrices(A and B). It typically is set between 8 to 256. Higher number means it can capture more diverse tasks but can lead to overfitting.
- **Alpha(α)**: Scaling factor applied to the matrices.
By now we already know
final output = base model output + LoRA update
But before adding the LoRA update, it is scaled

```
scaled LoRA update = LoRA update * (lora_alpha / r)
```

If α is too low:

LoRA learns, but its effect may be weak. The model may still behave too much like the base model.

If α is too high:

LoRA can overpower the base model. The model may become unstable, over-stylized, or worse at general reasoning.

So generally α is taken twice the size of r.



In our project related to Ayanokoji persona here is the value of these hyperparameters were

```
r = 16  
lora_alpha = 32
```


LoRA can be applied to the various parts of the model architecture. General advice is to add it in the attention layers.

```
target_modules = [  
    "q_proj", "k_proj", "v_proj", "o_proj",  
    "gate_proj", "up_proj", "down_proj",  
]
```

So in two major places which belong to attention mechanism

1. Attention layers
2. MLP / feed-forward layers

Attention decides:

Which previous tokens should the model focus on when producing the next token?



For example, if the user asks:

How should I handle betrayal?

The model needs to focus on concepts like:

betrayal trust emotion response strategy control

The attention layers help the model decide what matters in the prompt and conversation history.

The four pieces are:

```
q_proj = query projection
k_proj = key projection
v_proj = value projection
o_proj = output projection
```

Simple explanation:

```
q_proj: what am I looking for?
k_proj: what information exists?
v_proj: what information should I carry forward?
o_proj: how do I combine the attended information?
```

For our Ayanokoji model, attention layers help it notice the psychological or strategic angle instead of defaulting to generic comfort.

MLP / Feed-Forward Layers:

```
"gate_proj", "up_proj", "down_proj"
```

These belong to the model's feed-forward network, also called the MLP block.

After attention gathers context, the MLP transforms that information internally.

A simple way to think of it:

```
Attention = what should I look at?
MLP       = how should I think about it?
```

The MLP layers affect:

```
reasoning
style
tone
transformation
word
choice
abstraction
personality
patterns
```

The three pieces are:

```
gate_proj = controls what information passes through
up_proj   = expands information into a richer hidden space
down_proj = compresses it back into the model dimension
```

Because the persona is not only about attention. It is also about how the model transforms the user's question into an answer.

So that's why both were trained

```
Attention: decide what information matters.
MLP: process that information into meaning and style.
```

Now let's come to how we finetune the LLM to behave like Ayanokoji.

We took a llama-8b instruct model as base model capable of following instructions really well.

Then we finetuned this model to behave like Ayanokoji on 1000 SFT examples on how would ayanokoji answer questions. *Remember LORA only adds weights on top and never change base model weights.*

We used Unsloth which helped us finetune Llama 3.1 8B efficiently by loading the model in 4-bit and training only small LoRA adapters instead of the full model.

We used `FastLanguageModel.from_pretrained()` to load the base model, `get_peft_model()` to add LoRA adapters, and `use_gradient_checkpointing="unsloth"` to reduce VRAM usage.

Unsloth helps you finetune large models like Llama 3.1 8B with less VRAM and faster training.

Unsloth is doing four big jobs:

1. Loads the model efficiently
2. Enables 4-bit QLoRA training
3. Adds LoRA adapters cleanly
4. Saves VRAM with optimized gradient checkpointing

The official Unsloth docs describe it as a framework for faster fine-tuning with much lower VRAM usage, and their examples use the same pattern:

`FastLanguageModel.from_pretrained(...)` followed by

`FastLanguageModel.get_peft_model(...)`.

In short: Unsloth made SFT and DPO faster, lighter, and practical on a single GPU.

1. We first initialized and loaded the base model and tokenizer

```
import os
import unsloth
from unsloth import FastLanguageModel
from datasets import load_dataset
from transformers import TrainingArguments
from trl import SFTTrainer

MODEL_NAME      = "unsloth/Llama-3.1-8B-Instruct"
MAX_SEQ_LENGTH  = 2048
DATA_PATH       = "data/ayanokoji_finetune.jsonl"
OUTPUT_DIR      = "outputs/sft-ayanokoji"

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name      = MODEL_NAME,
    max_seq_length  = MAX_SEQ_LENGTH,
    dtype           = None,          # auto - bfloat16 on Ampere+, float16 elsewhere
```

```
load_in_4bit = True,      # QLoRA: saves ~50 % VRAM vs full precision
)
```

2. Then we added LoRA Adapters to the base model

```
model = FastLanguageModel.get_peft_model(
    model,
    r              = 16,      # rank – higher = more capacity, more
    lora_alpha     = 32,      # scaling factor (alpha / r = 2 is cor
    lora_dropout   = 0.05,    # slight dropout helps generalisation
    bias          = "none",
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    use_gradient_checkpointing = "unsloth",  # Unsloth's memory-efficient impl
    random_state              = 3407,
)
```

3. We chat templated the 1000 SFT examples we generated from Claude

```
def format_chat(example, tokenizer):
    """
    Apply the model's native chat template so every example becomes a
    single string with the correct special tokens for Llama-3.1 Instruct.
    add_generation_prompt=False because the assistant turn is already present.
    """
    text = tokenizer.apply_chat_template(
        example["messages"],
        tokenize = False,
        add_generation_prompt = False,
    )
    return {"text": text}

raw_ds = load_dataset("json", data_files=DATA_PATH, split="train")
split_ds = raw_ds.train_test_split(test_size=0.1, seed=42)

train_ds = split_ds["train"].map(
    lambda x: format_chat(x, tokenizer),
    remove_columns = split_ds["train"].column_names,
)
eval_ds = split_ds["test"].map(
```

```

lambda x: format_chat(x, tokenizer),
remove_columns = split_ds["test"].column_names,
)

print(f"Train samples : {len(train_ds)}")
print(f"Eval samples : {len(eval_ds)}")

```

4. We then define training arguments

```

args = TrainingArguments(
    output_dir = OUTPUT_DIR,
    # Effective batch = per_device * grad_accum = 2 * 4 = 8
    per_device_train_batch_size = 2,
    gradient_accumulation_steps = 4,
    num_train_epochs = 3,
    learning_rate = 2e-4,
    lr_scheduler_type = "cosine", # cosine > linear for persona task
    warmup_ratio = 0.05, # 5 % warmup prevents early instabilities
    # Logging & evaluation
    logging_steps = 10,
    eval_strategy = "steps",
    eval_steps = 50,
    save_strategy = "steps",
    save_steps = 50,
    save_total_limit = 2,
    load_best_model_at_end = True, # keeps the checkpoint with lowest dev loss
    metric_for_best_model = "eval_loss",
    # Precision & memory
    bf16 = True,
    # Reporting – swap "wandb" if you want W&B tracking
    report_to = "none",
)

```

5. Finally we started the training

```

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = train_ds,
    eval_dataset = eval_ds,
    dataset_text_field = "text",
    max_seq_length = MAX_SEQ_LENGTH,
)

```

```

        packing            = True,          # pack short sequences → more efficient GPU
        args               = args,

    )

    print("\n=== Starting SFT ===")
    trainer.train()

```

6. Once training is completed we save the new SFT trained model

```

trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)

```

Here now our model got ability to reply in Ayanokoji tone from the questions.

QLORA

This involves quantizing the base model parameters to a custom 4 bit NF4 data type, which significantly reduces memory usage.

In practice, QLoRA is LoRA applied on top of a quantized 4-bit base model, which greatly reduces VRAM usage while still training small adapter weights.

In lora if we set during loading base model

```
load_in_4bit = True
```

then it becomes QLORA 😊

So we loaded base model like this in our project

```

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name      = MODEL_NAME,
    max_seq_length  = MAX_SEQ_LENGTH,
    dtype           = None,          # auto - bfloat16 on Ampere+, float16 elsewhere

```

```
load_in_4bit = True,      # QLoRA: saves ~50 % VRAM vs full precision
)
```

So technically we used QLoRA

LoRA:

Base model stays in normal precision, usually FP16/BF16.

Only small adapter weights are trained.

QLoRA:

Base model is quantized to 4-bit.

Only small adapter weights are trained.

Much lower VRAM usage.

Preference Alignment FINE-TUNING

Preference Alignment is used because SFT teaches the model to imitate the target style, but it does not strongly teach preference between a good persona response and a generic one.

chosen = better Ayanokoji-style response

rejected = weaker/generic response

SFT teaches the model how to sound like Ayanokoji.

Preference Alignment teaches it which Ayanokoji-style response is preferred over

RLHF

It combines Reinforcement learning with Human feedback for inputs. We will not discuss this much as Direct Preference Optimization(DPO) is more preferred approach.

DPO

Prompt: “How do I become more productive?”

Two answers:

Chosen (better):

Break your work **into** small priorities, **remove** distractions, **and** focus **on** one in

Rejected (worse):

Just work harder than everyone **else**.

DPO's job is simple: *DPO trains the model to assign higher relative probability to the chosen response than the rejected response, while staying close to the reference model.*

Traditional RLHF vs DPO**RLHF:**

1. Train base model
2. Collect preference pairs
3. Train reward model ("which answer is better?")
4. Use PPO RL to maximize reward
5. Complex, expensive

DPO:

1. Train base model
2. Collect preference pairs
3. Directly optimize chosen > rejected

No reward model. No PPO.

Example:

Base Llama might answer: *"You should remain calm and assess."*

Our DPO-trained model should answer: *“Calmness preserves leverage. Assess motives before acting.”*

DPO checks: **Did we become better according to preferences without becoming nonsense?**

Hyperparameter: Beta (β)

β controls the strength of the preference optimization relative to the reference model. In practice, **value 0.1 is commonly used**, but the best value depends on the dataset and how aggressively you want the model to adapt.

Dataset Generation for DPO

There is no specific method to generate data for DPO mostly it's human generated or LLM generated. It's mostly in the format like

```
{
  "prompt": "How should I handle competition?",
  "chosen": "Competition is won before it begins—through preparation and position.",
  "rejected": "Just do your best and believe in yourself."
}
```

So it has prompt and then two answers to it one is preferred and another is rejected so in DPO training model learns to output more answer which are like chosen. So we are adding our preference on top of model like how it should answer.

DPO is like showing your model two answers and repeatedly saying: “This one. More like this.”

For our project I used Claude to generate around 200 DPO dataset pairs of chosen and rejected pairs for prompts.

DPO Fine-Tuning

DPO finetuning is done on top of the SFT finetuned model not the base model.

So we first finetune the base model using SFT and then finetune it further using DPO.

```
Final model = Base model + merged SFT behavior + DPO preference adapter
```

So in our case remember we saved the SFT model during LORA training now we will use DPO to finetune it further for preference alignment.

1. We load the base model and merge the SFT adapter weights learned during SFT training we did

```
import os
import torch
import unsloth
from unsloth import FastLanguageModel
from datasets import load_dataset
from trl import DPOTrainer, DPOConfig
from peft import PeftModel

# — Config —
BASE_MODEL_NAME = "unsloth/Llama-3.1-8B-Instruct"
SFT_OUTPUT_DIR = "outputs/sft-ayanokoji"      # ← adapter from Stage 1
DPO_OUTPUT_DIR = "outputs/dpo-ayanokoji"
DPO_DATA_PATH = "data/ayanokoji_dpo.jsonl"
MAX_SEQ_LENGTH = 2048

# — 1. Load base model + SFT adapter —
#
# We load the base model and then merge in the SFT LoRA weights.
# The merged model becomes the DPO *policy* (what we train).
# DPOTrainer will use a frozen copy as the *reference model* internally.
#
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = BASE_MODEL_NAME,
    max_seq_length = MAX_SEQ_LENGTH,
    dtype = None,
    load_in_4bit = True,
)

# Load and merge the SFT adapter weights into the base model
model = PeftModel.from_pretrained(model, SFT_OUTPUT_DIR)
model = model.merge_and_unload()      # merge LoRA → full weights (still 4-
```

2. Attaching fresh sets of lora adapters for DPO training

```
# Attach a fresh set of LoRA adapters for DPO training
model = FastLanguageModel.get_peft_model(
    model,
    r                = 8,          # smaller rank for DPO (fine-grained)
    lora_alpha       = 16,
    lora_dropout     = 0.05,
    bias             = "none",
    target_modules   = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    use_gradient_checkpointing = "unsloth",
    random_state     = 3407,
)
```

3. Create train and eval data on DPO dataset

[Open in app](#)

≡ **Medium**



b

```
print(f"DPO train pairs : {len(split_ds['train'])}")
print(f"DPO eval pairs : {len(split_ds['test'])}")
```

4. Set DPO config (observe the β hyperparameter)

```
dpo_config = DPOConfig(
    output_dir          = DPO_OUTPUT_DIR,
    beta                = 0.1,
    # Smaller batches + accumulation – DPO is more memory hungry than SFT
    per_device_train_batch_size = 1,
    gradient_accumulation_steps = 8,          # effective batch = 8
    num_train_epochs     = 2,          # 2–3 epochs is usually enough for DPO
    learning_rate        = 5e-5,      # lower than SFT – we're making fine-tune
    lr_scheduler_type    = "cosine",
    warmup_ratio         = 0.1,
    # Logging & eval
    logging_steps        = 5,
    eval_strategy        = "steps",
    eval_steps           = 20,
    save_strategy        = "steps",
    save_steps           = 20,
    save_total_limit     = 2,
```

```

load_best_model_at_end      = True,
# Sequence lengths for chosen/rejected
max_prompt_length          = 512,
max_length                  = MAX_SEQ_LENGTH,
# Precision
bf16                        = True,
report_to                   = "none",
)

```

4. Perform DPO training

```

trainer = DPOTrainer(
    model          = model,
    ref_model      = None,          # implicit reference from initial policy weights
    args           = dpo_config,
    train_dataset  = split_ds["train"],
    eval_dataset   = split_ds["test"],
    tokenizer      = tokenizer,
)

# — 5. Train —————
print("\n=== Starting DPO ===")
trainer.train()

```

5. Save the final model

```

trainer.model.save_pretrained(DPO_OUTPUT_DIR)
tokenizer.save_pretrained(DPO_OUTPUT_DIR)
print(f"\n✓ DPO adapter saved to: {DPO_OUTPUT_DIR}")

```

This final model is our LLM finetuned to give Ayanokoji persona.

To see the demo of the model check out my LinkedIn post

https://www.linkedin.com/posts/anubhav-mandarwal_i-really-wanted-to-enroll-but-couldnt-in-ugcPost-7458534465615732737-pnK5?utm_source=share&utm_medium=member_desktop&rcm=ACoAACvsMEQB79txviYP56p0j_vX1eObTiN8284

Evaluation: Checking Whether the Model Stayed in Persona

After SFT and DPO, I wanted to check whether the model was actually moving toward the Ayanokoji-style persona or simply producing generic assistant responses.

For this, I evaluated three signals:

1. Reward Margin
2. Style Score
3. Perplexity

The most important metric for this project was reward margin.

Reward Margin

The DPO dataset contains examples in this format:

```
{  
  "prompt": "How do I deal with failure?",  
  "chosen": "Failure is data. Nothing more. The emotional weight you attach to i  
  "rejected": "Failure is never the end. Believe in yourself and keep going!"  
}
```

Here, the chosen response is the preferred Ayanokoji-style answer, while the rejected response is a more generic motivational answer.

To calculate reward margin, I checked how likely the model thinks each response is.

Conceptually:

```
reward_margin =  $\log P(\text{chosen response} \mid \text{prompt}) - \log P(\text{rejected response} \mid \text{prompt})$ 
```

If the reward margin is positive, the model prefers the chosen Ayanokoji-style response.

If it is negative, the model prefers the rejected generic response.

So reward margin directly tells us whether the model is statistically leaning toward the persona we trained it for.

In my evaluation results:

```
Base model reward margin: -0.7013
SFT reward margin: 0.1161
DPO reward margin: 0.6352
```

This means the base model preferred generic responses, SFT shifted it toward the persona, and DPO made that preference much stronger.

FINAL TABLE

Technique	What it changes	Best for
Full fine-tuning	Original model weights	Deep task/domain adaptation
Partial fine-tuning	Some model layers	Small models or controlled
LoRA	Small adapter weights	Efficient behavior/task ada
QLoRA	LoRA + 4-bit base model	Fine-tuning large models or
DPO	Preference behavior	Making model prefer better

- Fine Tuning
- LLM
- Artificial Intelligence
- Machine Learning
- Transformers



Following

Published in Towards AI

139K followers · Last published 1 day ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Follow

Written by Anubhav Mandarwal

5 followers · 1 following

AI Engineering | SRE/Platform Engineering | Computer Science | Anime Philosophy

